SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.* (BL2, BL3)

Activity Tool : Case Study (Medium)
Assessment Tool Weightage: 30%

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.	BL2 – Understand	**5**
2	A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.	BL3 – Apply	**5**
3	Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.	BL3 – Apply	**5**
4	Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.	BL3 – Apply	**5**
5	Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.	BL2 – Understand	**5**
6	Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.	BL3 – Apply	**5**

7	Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.	BL2 – Understand	**5**
8	A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.	BL3 – Apply	**5**
9	Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.	BL3 – Apply	**5**
10	Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.	BL3 – Apply	**5**

Code of Conduct:

I Darshinie Karani.V.M (192520033) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning

Solution Design	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

ANSWERS

1. Identify Functional Dependencies

From the given attributes:

1. OrderID → CustID
2. CustID → CustName
3. ProdID → ProdName, Price
4. (OrderID, ProdID) → Qty

So,

FD set:

F = { OrderID → CustID, CustID → CustName, ProdID → ProdName, Price, (OrderID, ProdID) → Qty }

Find Candidate Key

- We need an attribute set that determines all attributes.
- Take:
    - (OrderID, ProdID)
- Using the FDs:
    - OrderID → CustID
    - CustID → CustName
    - ProdID → ProdName, Price
    - (OrderID, ProdID) → Qty

Therefore:

(OrderID, ProdID) → OrderID, ProdID, CustID, CustName, ProdName, Qty, Price

Hence, Candidate Key = (OrderID, ProdID)

Check 1NF

The relation is already in 1NF because all attributes contain atomic/single values.

Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

Convert to 2NF

A relation is in 2NF when there are no partial dependencies on a part of the composite key.

Candidate key:

(OrderID, ProdID)

But:

OrderID → CustID

CustID → CustName

ProdID → ProdName, Price

These attributes depend only on part of the key.

So, decompose into:

Customer

Customer(CustID, CustName)

PK: CustID

Product

Product(ProdID, ProdName, Price)

PK: ProdID

Order

Order(OrderID, CustID)

PK: OrderID

OrderDetails

OrderDetails(OrderID, ProdID, Qty)

PK: (OrderID, ProdID)

Check 3NF

In the original relation:

OrderID → CustID

CustID → CustName

Therefore:

OrderID → CustName

- This is a transitive dependency.

- We already removed it by creating the Customer relation.
- Now the relations are:
    - Customer(CustID, CustName)
    - Product(ProdID, ProdName, Price)
    - Order(OrderID, CustID)
    - OrderDetails(OrderID, ProdID, Qty)

Relation	Primary Key	Attributes
Customer	CustID	CustID, CustName
Product	ProdID	ProdID, ProdName, Price
Order	OrderID	OrderID, CustID
OrderDetails	OrderID, ProdID	OrderID, ProdID, Qty

2.

Identify Functional Dependencies

From the relation:

1. PID → PName, DocID, WardNo
2. DocID → DocName, Specialization
3. WardNo → WardName

So the FD set is:

F = { PID → PName, DocID, WardNo; DocID → DocName, Specialization; WardNo → WardName }

Find Candidate Key

Take PID, Using the dependencies:

PID → PName, DocID, WardNo

Then:

DocID → DocName, Specialization and WardNo → WardName

Therefore:

PID → PName, DocID, WardNo, DocName, Specialization, WardName

Hence,Candidate Key = PID

Identify Anomalies

1. Update Anomaly

If a doctor's name or specialization changes, we need to update it in multiple patient records.

2. Insertion Anomaly

A new doctor cannot be added unless a patient is assigned to that doctor.

3. Deletion Anomaly

If the last patient of a doctor is deleted, the doctor's information may also be lost.

Check BCNF

A relation is in BCNF if for every functional dependency X → Y, X must be a superkey.

Here:

PID → PName, DocID, WardNo
PID is a candidate key.

But:

DocID → DocName, Specialization
DocID is not a key of the original relation.

Also:

WardNo → WardName
WardNo is not a key of the original relation.

Therefore, the original relation violates BCNF.

Decompose into BCNF

1. Patient Relation

Patient(PID, PName, DocID, WardNo)

Primary Key: PID

FD: PID → PName, DocID, WardNo

2. Doctor Relation

Doctor(DocID, DocName, Specialization)

Primary Key: DocID

FD: DocID → DocName, Specialization

3. Ward Relation

Ward(WardNo, WardName)

Primary Key: WardNo

FD: WardNo → WardName

Final BCNF Schema

Patient(PID, PName, DocID, WardNo)
Doctor(DocID, DocName, Specialization)
Ward(WardNo, WardName)

Relation	Primary Key
Patient	PID
Doctor	DocID
Ward	WardNo

3.

Functional Dependencies:

- SID → SName
- CourseID → CourseName, Faculty
- (SID, CourseID) → Grade

Candidate Key:

The candidate key is (SID, CourseID)

Because (SID, CourseID) determines all the attributes in the relation.

Check for 1NF:

The relation is in 1NF because all attributes contain atomic values.

Partial Dependencies:

Since the candidate key is (SID, CourseID):

- SID → SName is a partial dependency.
- CourseID → CourseName, Faculty is a partial dependency.

Therefore, the relation is not in 2NF.

2NF Decomposition:

Student(SID, SName)
Primary Key: SID

Course(CourseID, CourseName, Faculty)
Primary Key: CourseID

Enrollment(SID, CourseID, Grade)
Primary Key: (SID, CourseID)

Final Answer:
Student(SID, SName)
Course(CourseID, CourseName, Faculty)
Enrollment(SID, CourseID, Grade)

Thus, the original Enrollment relation is decomposed into 2NF by removing all partial dependencies.

4.

Functional Dependencies

- PassID → PassName
- FlightNo → DeptCity, ArrCity, Date
- (FlightNo, PassID) → Seat

Candidate Key

Candidate Key = (FlightNo, PassID)

Because (FlightNo, PassID) determines all the attributes in the relation.

1NF

The relation is in 1NF because all attributes contain atomic values.

BCNF Check

For BCNF, for every functional dependency X → Y, X must be a superkey.

- PassID → PassName — Violation, because PassID is not a superkey.
- FlightNo → DeptCity, ArrCity, Date — Violation, because FlightNo is not a superkey.
- (FlightNo, PassID) → Seat — Satisfies BCNF because (FlightNo, PassID) is the candidate key.

Therefore, the original relation is not in BCNF.

BCNF Decomposition

Passenger(PassID, PassName)

Primary Key: PassID

Flight(FlightNo, DeptCity, ArrCity, Date)

Primary Key: FlightNo

Booking(FlightNo, PassID, Seat)

Primary Key: (FlightNo, PassID)

Final Answer: Passenger(PassID, PassName)

Flight(FlightNo, DeptCity, ArrCity, Date)

Booking(FlightNo, PassID, Seat)

Conclusion

The original relation is decomposed into Passenger, Flight, and Booking relations. In each decomposed relation, the determinant is a superkey. Hence, the final relations are in BCNF.

5.

Functional Dependencies

The functional dependencies are:

- EmpID → EmpName, DeptID, ManagerID
- DeptID → DeptName, ManagerID

- ManagerID → ManagerName

This means:

- Each employee ID identifies the employee name, department, and manager.
- Each department ID identifies the department name and manager.
- Each manager ID identifies the manager name.

Candidate Key

Candidate Key = EmpID

Because EmpID → EmpName, DeptID, ManagerID

DeptID → DeptName

ManagerID → ManagerName

Therefore, EmpID determines all attributes.

Check 1NF

The relation is in 1NF because all attributes contain single and atomic values.

Check 2NF

The candidate key is a single attribute, EmpID.

Therefore, there cannot be any partial dependency.

Hence, the relation is in 2NF.

Identify Transitive Dependencies

We have:

EmpID → DeptID

DeptID → DeptName

Therefore:

EmpID → DeptName

This is a transitive dependency.

Similarly:

EmpID → ManagerID

ManagerID → ManagerName

Therefore:

EmpID → ManagerName

This is also a transitive dependency.

So, the relation is not in 3NF.

Decomposition into 3NF

Employee:

Employee(EmpID, EmpName, DeptID, ManagerID)

Primary Key: EmpID

Department:

Department(DeptID, DeptName, ManagerID)

Primary Key: DeptID

Manager:

Manager(ManagerID, ManagerName)

Primary Key: ManagerID

Final 3NF Relations

Employee(EmpID, EmpName, DeptID, ManagerID)

Department(DeptID, DeptName, ManagerID)

Manager(ManagerID, ManagerName)

Conclusion

The original relation contains transitive dependencies:

EmpID → DeptID → DeptName

EmpID → ManagerID → ManagerName

By decomposing the relation into Employee, Department, and Manager, the transitive dependencies are removed. Therefore, the resulting relations satisfy 3NF.

6.

Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate)

Identify the Multivalued Dependency (MVD)

A member can borrow many books, so one MemberID can be associated with multiple BookIDs.

Therefore, the MVD is:

MemberID →→ BookID

The book details such as Title, Author, and Genre depend on the BookID, while BorrowDate depends on the borrowing of a particular book by a member.

Why does it violate 4NF?

A relation is in 4NF if, for every non-trivial MVD X →→ Y, X is a super key.

Here:

MemberID →→ BookID

But MemberID alone is not a super key of the Library relation because one member can have many books.

Hence, the relation violates 4NF.

Decompose the Relation

We divide the relation into three relations:

MEMBER

MEMBER(MemberID, MemberName)

Primary Key: MemberID

BOOK

BOOK(BookID, Title, Author, Genre)

Primary Key: BookID

BORROW

BORROW(MemberID, BookID, BorrowDate)

Primary Key: (MemberID, BookID)

MemberID → Foreign Key

BookID → Foreign Key

Result

The final 4NF decomposition is:

Library → MEMBER + BOOK + BORROW

MEMBER

MemberID (PK)

MemberName

BOOK

BookID (PK)

Title

Author

Genre

BORROW

MemberID (PK, FK)

BookID (PK, FK)

BorrowDate

Conclusion

The original Library relation contains the MVD MemberID →→ BookID, which causes redundancy. By decomposing it into MEMBER, BOOK, and BORROW, the multivalued dependency is separated and the relations satisfy 4NF.

7.

TELECOM_BILLING(CustomerID, CustomerName, PhoneNo, PlanID, PlanName, BillNo, BillDate, Amount)

Functional Dependencies

The functional dependencies are:

1. CustomerID → CustomerName, PhoneNo

2. PhoneNo → CustomerID, CustomerName

3. PlanID → PlanName

4. BillNo → CustomerID, PlanID, BillDate, Amount

This means:

- CustomerID determines the customer's name and phone number.
- PhoneNo identifies a particular customer.
- PlanID determines the plan name.
- BillNo uniquely identifies a bill and determines its details.

Candidate Keys

A candidate key is an attribute or set of attributes that uniquely identifies each tuple.

Here, BillNo uniquely identifies a bill.

Therefore:

Candidate Key = {BillNo}

Primary Key

Since BillNo is the candidate key, we select it as the primary key.

Primary Key = BillNo

Summary Table

Functional Dependency Meaning

CustomerID → CustomerName, PhoneNo Customer details

PhoneNo → CustomerID, CustomerName Phone identifies customer

PlanID → PlanName Plan details

BillNo → CustomerID, PlanID, BillDate, Amount Bill details

Final Answer

Candidate Key: BillNo

Primary Key: BillNo

Functional Dependencies:

CustomerID → CustomerName, PhoneNo

PhoneNo → CustomerID, CustomerName

PlanID → PlanName

BillNo → CustomerID, PlanID, BillDate, Amount

Thus, the telecom billing schema has BillNo as its candidate key and primary key, with the above functional dependencies.

8.

Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)

Functional Dependencies

The main functional dependencies are:

PID → PName, SupplierID, Category, Price, Warehouse

SupplierID → SupplierName

Therefore, there is a transitive dependency:

PID → SupplierID → SupplierName

First Normal Form (1NF)

The relation is in 1NF because:

All attributes contain atomic values.

There are no repeating groups.

Each row can be uniquely identified by PID.

So:

Product is in 1NF.

Second Normal Form (2NF)

The primary key is PID, which is a single attribute.

Therefore, partial dependency is not possible.

So:

Product is in 2NF.

Third Normal Form (3NF)

There is a transitive dependency:

PID → SupplierID

and

SupplierID → SupplierName

Therefore:

PID → SupplierName

This violates 3NF because SupplierName depends indirectly on the primary key through SupplierID.

Decomposition

Split the relation into two relations.

PRODUCT

PRODUCT(PID, PName, SupplierID, Category, Price, Warehouse)

Primary Key: PID

Foreign Key: SupplierID

SUPPLIER

SUPPLIER(SupplierID, SupplierName)

Primary Key: SupplierID

Final Normalized Schema

PRODUCT

PID (PK)

PName

SupplierID (FK)

Category

Price

Warehouse

SUPPLIER

SupplierID (PK)

SupplierName

Conclusion

The original relation is initially in 1NF and 2NF, but violates 3NF due to the transitive dependency:

PID → SupplierID → SupplierName

After decomposition into PRODUCT and SUPPLIER, the design achieves 3NF and, under the given dependencies, BCNF as well.

9.

SCHOOL(StudentID, StudentName, CourseID, CourseName, TeacherID, TeacherName, DepartmentID, DepartmentName, Grade)

Functional Dependencies

The functional dependencies are:

1. StudentID → StudentName

2. CourseID → CourseName, TeacherID

3. TeacherID → TeacherName, DepartmentID

4. DepartmentID → DepartmentName

5. (StudentID, CourseID) → Grade

The candidate key of the original relation is:

(StudentID, CourseID)

because a student's grade is identified by the combination of the student and course.

Problems in the Original Relation

There are partial dependencies:

StudentID → StudentName

CourseID → CourseName, TeacherID

There are also transitive dependencies:

CourseID → TeacherID → TeacherName

TeacherID → DepartmentID → DepartmentName

Therefore, the relation is not in 2NF or 3NF.

Decomposition

STUDENT

STUDENT(StudentID, StudentName)

Primary Key: StudentID

COURSE

COURSE(CourseID, CourseName, TeacherID)

Primary Key: CourseID

Foreign Key: TeacherID

TEACHER

TEACHER(TeacherID, TeacherName, DepartmentID)

Primary Key: TeacherID

Foreign Key: DepartmentID

DEPARTMENT

DEPARTMENT(DepartmentID, DepartmentName)

Primary Key: DepartmentID

ENROLLMENT

ENROLLMENT(StudentID, CourseID, Grade)

Primary Key: (StudentID, CourseID)

Foreign Keys: StudentID, CourseID

Final Normalized Schema

STUDENT

StudentID (PK)

StudentName

COURSE

CourseID (PK)

CourseName

TeacherID (FK)

TEACHER

TeacherID (PK)

TeacherName

DepartmentID (FK)

DEPARTMENT

DepartmentID (PK)

DepartmentName

ENROLLMENT

StudentID (PK, FK)

CourseID (PK, FK)

Grade

BCNF Justification

In each decomposed relation, every determinant is a candidate key:

StudentID → StudentName

CourseID → CourseName, TeacherID

TeacherID → TeacherName, DepartmentID

DepartmentID → DepartmentName

(StudentID, CourseID) → Grade

Therefore, all the decomposed relations satisfy BCNF.

Conclusion

The original School ERP relation contains partial and transitive dependencies, causing redundancy and update anomalies. By decomposing it into STUDENT, COURSE, TEACHER, DEPARTMENT, and ENROLLMENT, the database is normalized up to BCNF.

10.

STREAM(MovieID, ActorID, DirectorID)

A movie can have multiple actors and multiple directors. If these relationships are independent, the relation may contain redundant combinations.

Join Dependency

The join dependency can be represented as:

(MovieID, ActorID), (MovieID, DirectorID), (ActorID, DirectorID)

This means the original relation can be reconstructed by joining the smaller relations.

Decomposition

Decompose the relation into:

MOVIE_ACTOR(MovieID, ActorID)

Primary Key: (MovieID, ActorID)

MOVIE_DIRECTOR(MovieID, DirectorID)

Primary Key: (MovieID, DirectorID)

ACTOR_DIRECTOR(ActorID, DirectorID)

Primary Key: (ActorID, DirectorID)

Lossless Join

The original relation can be reconstructed as:

STREAM = MOVIE_ACTOR ⋈ MOVIE_DIRECTOR ⋈ ACTOR_DIRECTOR

Therefore, the decomposition is lossless if the stated join dependency is a valid business rule.

5NF Justification

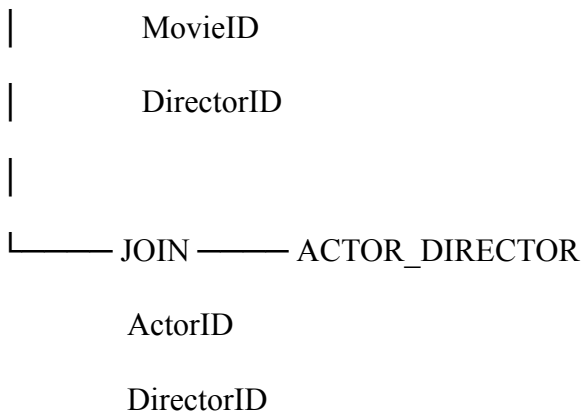
A relation is in 5NF when every non-trivial join dependency is implied by the candidate keys.

After decomposition, the independent relationships are stored separately:

MOVIE_ACTOR

MovieID

ActorID



Thus, the decomposition removes the non-trivial join dependency from the original relation and produces a 5NF design.

Original:

STREAM(MovieID, ActorID, DirectorID)

5NF Relations:

1. MOVIE_ACTOR(MovieID, ActorID)

2. MOVIE_DIRECTOR(MovieID, DirectorID)

3. ACTOR_DIRECTOR(ActorID, DirectorID)

Lossless Join:

MOVIE_ACTOR ⋈ MOVIE_DIRECTOR ⋈ ACTOR_DIRECTOR = STREAM

Hence, the decomposition is lossless and in 5NF, assuming the stated join dependency accurately represents the movie-streaming business rules.